JEV / SYSTEM ONE

HANDWRITTEN-STYLE MASTER NOTES

System One • Typed Decisions • Probability • Confidence • Calibration • Parallel Sampling

Study-focused notebook version of the supplied master notes.

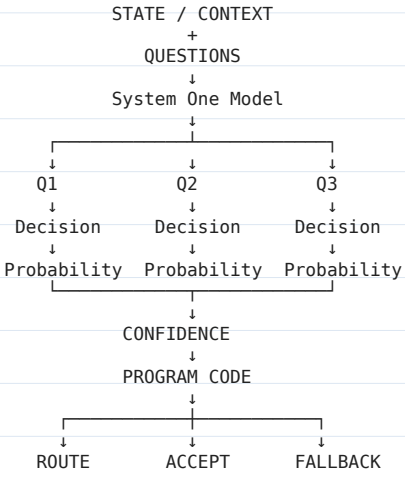
JEV / SYSTEM ONE — HANDWRITTEN-STYLE MASTER NOTES

0. First understand the BIG picture

Traditional LLM



Jev / System One



[KEY] Core idea

> *LLM → generates language*
>
> *Jev → makes structured decisions*

TypeSafe describes Jev as a System One model designed around typed decisions, probabilities, confidence, and parallel sampling rather than ordinary autoregressive text generation. ([arihantbooks](https://arihantbooks.com/products/computer-knowledge-capsule-handwritten-notes-ssc-railway-police-banking-afcat-csat?variant=44121938985005\&utm_source=chatgpt.com "Computer Knowledge Capsule Book 2026 Handwritten Notes | Hindi Medium – arihantbooks"))

---

1. What is Jev?

Definition

Jev is TypeSafe AI's System One decision model.

Instead of asking:

> *"What text should I generate next?"*

we ask:

> *"Given this state, which decision should I make?"*

Example

Suppose:

```
Customer:
"My payment was deducted but my order was not placed."
```

Traditional LLM:

```
→ generate explanation
→ generate response
→ generate JSON
```

Jev-style decision:

```
Question:
Which department?

Options:
[Billing, Technical, Delivery]

Output:
Billing → 0.91
Technical → 0.06
Delivery → 0.03
```

Then normal software can do:

```
if decision == "Billing":
    route_to_billing()
```

[KEY] Remember

```
LLM → "Tell me something."

Jev → "Choose something."
```

---

2. Why was a model like Jev needed?

Traditional LLMs are extremely powerful.

But imagine your application only needs:

```
YES / NO
```

or

```
A / B / C
```

or

```
LOW / MEDIUM / HIGH
```

Generating a long natural-language response is unnecessary.

Example

You ask an LLM:

> *Is this email spam?*

It might answer:

> *“Based on the content of the email, it appears to have characteristics commonly associated with spam…”*

But your program only needs:

```
{
  "spam": true
}
```

So the idea becomes:

```
             Need a decision?
                    ↓
       Don't generate unnecessary text
                    ↓
             Make decision
                    ↓
        Give structured result
```

---

3. System One vs System Two

This is one of the most important concepts.

The terminology is inspired by the System 1/System 2 distinction popularized by Daniel Kahneman.

For AI architecture:

System One

```
FAST
 ↓
DECIDE
 ↓
ROUTE
 ↓
ACT
```

System Two

```
THINK
 ↓
REASON
 ↓
PLAN
 ↓
GENERATE
```

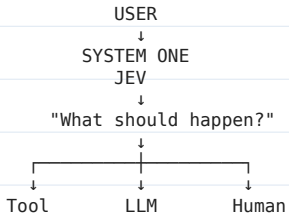
Comparison

System One / Jev	Generative LLM
Fast decisions	Deep generation/reasoning
Structured output	Natural language
Typed choices	Open-ended text
Parallel decisions	Autoregressive tokens
Routing	Writing
Classification	Explanation
Guardrails	Complex reasoning

[KEY] Important

System One does NOT necessarily replace System Two.

A better architecture can be:



So:

> *System One decides what to do.*
>
> *System Two does difficult reasoning when required.*

---

4. State

What is State?

State = information available to the model when making a decision.

Examples:

```
Email
Customer ticket
User message
JSON
Browser state
Application state
```

```
Tool call
Game state
Document
```

Example

```
STATE:

Customer:
"I paid ₹2000 but order is still pending."

Previous action:
Payment completed

Order:
Pending
```

This becomes the input state.

---

5. Question

The application tells the model:

> *What exactly do I want to know?*

Example:

```
STATE
 ↓
Customer complaint
 ↓
QUESTIONS
 ↓
Q1 → Is this urgent?
Q2 → Which department?
Q3 → What severity?
```

So:

```
State = WHAT information do you have?

Question = WHAT decision do you want?
```

[KEY] Very important interview point.

---

6. Typed Decisions

Traditional LLM:

```
INPUT
 ↓
TEXT
 ↓
TEXT
 ↓
TEXT
```

Decision model:

```
INPUT
 ↓
DEFINED SCHEMA
 ↓
VALID ANSWER
```

Suppose:

```
Department =
[
 Billing,
 Technical,
 Sales
]
```

The answer space is already defined.

The model should choose from:

```
Billing
Technical
Sales
```

rather than inventing:

```
Customer Financial Resolution Team
```

[KEY] Key idea

> *The application defines the legal output space.*

---

7. Noul

One of the decision primitives discussed for Jev is Noul.

Think:

```
Noul
 ↓
YES / NO
```

Example

```
Question:

"Is this message spam?"

Output:

YES → 0.92
NO  → 0.08
```

Use cases

```
Is user authenticated?
Is content unsafe?
Should we escalate?
Is payment suspicious?
Is this tool call allowed?
```

---

8. Choice

Choice means:

> *Select one option from multiple possibilities.*

Example:

```
Question:

Which department?

[Billing]
[Technical]
[Sales]
```

Possible output:

```
Billing     0.80
Technical   0.15
Sales       0.05
```

Prediction:

```
Billing
```

because:

```
max(probability)
```

---

9. Score

Score means estimating a level/value across an ordered scale.

Example:

```
Question:

How serious is this complaint?

LOW
MEDIUM
```

```
HIGH
```

Output:

```
LOW       → 0.10
MEDIUM    → 0.25
HIGH      → 0.65
```

Prediction:

```
HIGH
```

---

10. Logits

Before Softmax, neural networks generally produce logits.

Example:

```
Billing     = 3.2
Technical   = 1.8
Sales       = 0.5
```

These are not probabilities yet.

They can be positive, negative, large or small.

Then:

```
LOGITS
   ↓
SOFTMAX
   ↓
PROBABILITIES
```

---

11. Softmax [KEY]

Softmax converts logits into probabilities.

Formula:

```
P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}
```

Where:

```
zᵢ = logit
Pᵢ = probability
```

Example

```
Logits:

Billing      3.2
Technical    1.8
Sales        0.5

          ↓
       Softmax

Billing      0.80
Technical    0.15
Sales        0.05
```

Important:

```
0 ≤ P ≤ 1

Sum of probabilities = 1
```

Memory trick

Logits = raw scores
Softmax = converts raw scores → probabilities

---

12. Probability vs Confidence

Don't confuse these.

Probability

Distribution over possible outcomes.

```
Billing      0.80
Technical    0.15
Sales        0.05
```

Confidence

How certain the model is about its decision.

Conceptually:

```
Decision = Billing
Confidence = 0.91
```

But do not automatically assume that every API's "confidence" is simply the maximum Softmax probability.

The exact definition depends on the model/API.

---

13. Calibration [KEY][KEY][KEY]

This is extremely important for Jev.

Suppose a model makes:

```
100 predictions
```

and for all of them it says:

```
Confidence = 80%
```

If it is well calibrated:

```
≈ 80 predictions should be correct
```

over a sufficiently large population.

Simple definition

> *Calibration = predicted confidence should correspond to actual observed correctness.*

Accuracy vs Calibration

Accuracy

Question:

> *Did I get the answer correct?*

Example:

```
100 predictions
90 correct

Accuracy = 90%
```

Calibration

Question:

> *Does my confidence mean what I say it means?*

Example:

```
Model says 80%
```

If only 50% are correct:

```
BAD calibration
```

If approximately 80% are correct:

```
GOOD calibration
```

---

14. Why confidence matters

Imagine an AI system deciding:

```
Should we automatically approve?
```

If:

```
confidence = 0.97
```

perhaps your application allows automatic processing.

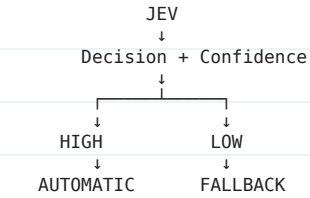
But:

```
confidence = 0.52
```

could trigger:

```
HUMAN REVIEW
```

Architecture:



This is called confidence-gated / selective automation as a general engineering pattern.

---

15. ECE — Expected Calibration Error

A common calibration metric is:

ECE = Expected Calibration Error

Basic intuition:

```
predicted confidence
          VS
actual accuracy
```

If the model says:

```
90%
```

but actual accuracy is:

```
65%
```

there is a calibration gap.

You don't need to memorize the complete implementation initially.

Remember:

> *ECE measures the gap between confidence and actual accuracy across confidence groups/bins.*

---

16. Brier Score

Another calibration/probabilistic prediction metric.

For binary classification:

```
Brier = \frac{1}{N}\sum_{i=1}^{N}(p_i-y_i)^2
```

where:

```
pᵢ = predicted probability
yᵢ = actual result
```

Lower is generally better.

---

17. Autoregressive LLM

This is critical to understand before Jev architecture.

Suppose the model must generate:

```
"I love AI"
```

It doesn't normally generate all three tokens independently.

Conceptually:

```
Prompt
 ↓
Token 1 = I
 ↓
Token 2 = love
 ↓
Token 3 = AI
```

So:

```
Token 2 depends on Token 1
Token 3 depends on Token 1 + Token 2
```

Mathematically:

```
P(y_1,y_2,y_3) = P(y_1) P(y_2|y_1) P(y_3|y_1,y_2)
```

This is autoregressive generation.

---

18. Why autoregressive generation can be slow

Imagine:

```
100 tokens
```

The model has a sequential generation process.

Conceptually:

```
1 → 2 → 3 → 4 → 5 → ... → 100
```

Even though GPU operations within a step can be highly parallelized, the dependency between generated tokens remains.

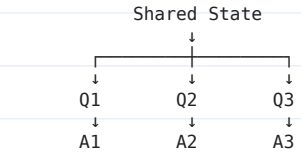
---

19. Non-autoregressive idea

Instead of:

```
Decision 1
    ↓
Decision 2
    ↓
Decision 3
    ↓
Decision 4
```

we want:



[KEY] Jev's important idea

Parallel decision sampling.

Multiple requested decisions can be evaluated using shared state rather than running a long token-generation chain.

---

20. Prefill

In normal LLM inference:

```
PROMPT
  ↓
```

```
PREFILL
  ↓
KV CACHE
  ↓
DECODE
```

Prefill

The model processes the existing input/context.

For example:

```
"Explain machine learning in simple words..."
```

The entire prompt is processed to establish the internal representation needed for generation.

---

21. Decode

After prefill:

```
Decode
 ↓
Token 1
 ↓
Token 2
 ↓
Token 3
 ↓
...
```

The decoding process generates output tokens sequentially.

---

22. Traditional LLM Architecture

Simplified:

```
Input
 ↓
Tokenizer
 ↓
Token Embeddings
 ↓
Transformer
 ↓
Hidden States
 ↓
LM Head
 ↓
Vocabulary Logits
 ↓
Softmax
 ↓
Next Token
```

LM Head

The LM head maps the model's hidden representation to scores over the vocabulary.

For example, vocabulary could contain:

```
the
cat
dog
is
running
...
```

So:

```
Hidden State
      ↓
    LM Head
      ↓
scores for thousands/millions of possible tokens
```

---

23. Your “Answer Head” concept

This is where your lecture becomes interesting.

Traditional

```
Hidden State
     ↓
   LM Head
     ↓
Vocabulary
     ↓
Next token
```

Lecture/reconstruction

```
Hidden State
     ↓
 Answer Head
     ↓
Allowed Answers
     ↓
Softmax / scoring
     ↓
Decision
```

[KEY] Main idea

Instead of asking:

> *"Which vocabulary token comes next?"*

the decision system asks:

> *"Which answer from the defined decision space should be selected?"*

---

24. IMPORTANT: Answer Head status

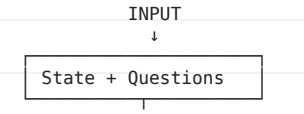
Write this in your notebook with a [KEY]:

> *Answer Head is part of the lecture/reverse-engineered conceptual architecture. It should NOT currently be described as a fully published official TypeSafe layer-by-layer architecture.*

This distinction prevents hallucination.

---

25. Jev architecture — study reconstruction




```
                INPUT
                  ↓
                  ↓
            Transformer
                  ↓
               Decoder
                  ↓
        Shared Representation
                  ↓
               Answer Head
               /          \
              ↓            ↓
             Q1            Q2
              ↓            ↓
      Answer Score   Answer Score
              └──────┬─────┘
                     ↓
                  Softmax
                     ↓
       Probabilities / Scores
                     ↓
              Confidence
                     ↓
               PROGRAM
```

Again:

This is your study/reconstruction model, not an official TypeSafe architecture diagram.

---

26. Synthetic Data

Definition

Synthetic data = artificially generated training examples.

Instead of manually collecting:

```
1,000,000 real examples
```

we can generate examples programmatically or using another model/system.

Conceptually:

```
Task definitions
      ↓
Synthetic examples
      ↓
Training
      ↓
Model
```

---

27. Why synthetic data?

Advantages:

```
✓ Large scale
✓ Cheap generation
✓ Controlled examples
✓ Edge cases
✓ Different schemas
✓ Different questions
✓ Rare scenarios
```

Example:

```
Task:
Detect whether a tool call is dangerous.

Generate:

safe example
safe example
dangerous example
borderline example
ambiguous example
...
```

---

28. Synthetic data limitation

Very important:

```
Synthetic Data
      ↓
Generated by some process
      ↓
Process has assumptions
      ↓
Training data inherits assumptions
```

So:

> *Synthetic ≠ automatically perfect.*

You still need real-world/held-out evaluation.

---

29. RLCD [KEY][KEY][KEY]

Your lecture highlights:

RLCD = Reinforcement Learning for Calibrated Decisions

The central idea is to train the model toward better-calibrated decisions, rather than only optimizing free-form language quality.

Conceptual flow:

```
Training examples
      ↓
Model prediction
      ↓
Probability / confidence
      ↓
Compare with outcome
      ↓
Reward / learning signal
      ↓
Model update
      ↓
Better calibrated decisions
```

Don't confuse

```
RLHF
 ↓
Human preference

RLVR
 ↓
Verifiable reward

RLCD
 ↓
Calibrated decisions
```

The exact internal RLCD training recipe is not fully public.

---

30. Why RLCD is important for System One

Suppose:

```
Decision = "Technical"

Confidence = 99%
```

but the model is often wrong.

Then confidence is not useful.

We want:

```
Confidence
     ↓
meaningful signal
     ↓
software can trust it statistically
```

Hence:

```
Decision
   +
Calibration
   ↓
Safe automation
```

---

31. “Jev cannot hallucinate” — correct understanding [KEY][KEY][KEY]

Don't write:

X

> *Jev cannot hallucinate.*

Write:

OK

> *Jev's schema-constrained output space can prevent invalid/unexpected output types, but it can still make an incorrect semantic decision.*

Example

Allowed:

```
A = Billing
B = Technical
C = Sales
```

Model predicts:

```
Billing
```

But correct answer was:

```
Technical
```

The model did not violate the schema.

It was simply wrong.

Therefore:

```
Schema safety
     ≠
Semantic correctness
```

---

32. Traditional JSON prompting vs typed decision

LLM approach

```
Prompt:

Return JSON.

{
  "department": "...",
  "confidence": ...
}
```

Potential issue:

```
Wrong structure
Missing field
Extra field
Wrong datatype
Invalid JSON
```

Typed decision approach

```
Schema:

department:
  Billing
  Technical
  Sales
```

The output space is constrained.

---

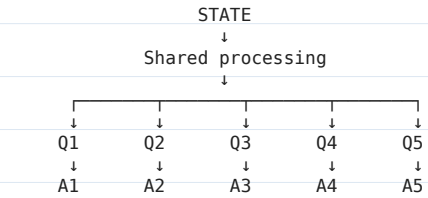
33. Parallel Questions

Suppose we need:

```
Q1 → Is spam?
Q2 → Which category?
Q3 → Is urgent?
Q4 → Severity?
Q5 → Which agent?
```

Traditional approach could involve multiple model calls / generation paths.

Conceptual Jev approach:



[KEY] Key phrase

> *Many decisions from shared state.*

---

34. Agent Routing

This is one of the most useful production applications.

Imagine:

```
User:
"Can you cancel my order?"
```

Jev:

```
Question:
Which agent?

[Billing]
[Order]
[Technical]
[Sales]
```

Result:

```
Order = 0.94
```

Code:

```
if agent == "Order":
    order_agent()
```

Architecture:

```
User
 ↓
Jev
 ↓
Route
 ↓
Specialized Agent
```

---

35. Guardrails

Suppose an agent wants to call:

```
delete_account()
```

Before executing:

```
Tool call
   ↓
Jev / decision model
   ↓
Allowed?
   ↓
YES → execute
NO  → block
```

This creates an AI guardrail layer.

---

36. Moderation

Input:

```
User content
```

Questions:

```
Is unsafe?
Which category?
Severity?
```

Output:

```
unsafe = YES
category = X
severity = HIGH
confidence = 0.93
```

Then:

```
HIGH confidence
       ↓
      BLOCK

LOW confidence
       ↓
     REVIEW
```

---

37. Real-time UI

Imagine a user interacts with an application.

System state:

```
Current page
User action
User history
Current task
Available tools
```

Decision model:

```
What should UI do next?
```

Possible choices:

```
SHOW_HELP
SHOW_FORM
ASK_CONFIRMATION
CALL_TOOL
SHOW_RESULT
```

Then code executes the selected UI action.

---

38. E-commerce example [KEY]

Review:

> *“The product arrived quickly, but the battery stopped working after two days. Support replaced it immediately.”*

Questions:

```
Q1 → Negative sentiment?
Q2 → Main issue?
Q3 → Urgency?
Q4 → Recommendation?
```

Possible output:

```
Q1:
YES → 0.88

Q2:
Battery → 0.91
Delivery → 0.03
Quality → 0.04
Other → 0.02

Q3:
High → 0.61
Medium → 0.30
Low → 0.09
```

Then business logic:

```
Battery issue
     +
High urgency
     ↓
Priority support
```

---

39. Cost concept

If a system generates a lot of text unnecessarily:

```
Input
 ↓
```

```
Reasoning
 ↓
100 tokens
 ↓
200 tokens
 ↓
500 tokens
 ↓
Final decision
```

But the application actually needs:

```
YES / NO
```

then much of that generation is unnecessary.

System One attempts to make the model interface more like:

```
Input
 ↓
Decision
```

TypeSafe's public material reports very low latency/cost figures, but treat those as vendor benchmark/price claims rather than universal guarantees. Always benchmark your own workload.

---

40. Jev vs LLM — Master table

Feature	Traditional LLM	Jev / System One
Main purpose	Generate language	Make decisions
Output	Tokens/text	Typed decisions
Generation	Autoregressive	Parallel decision sampling
Output space	Vocabulary	Defined schema/options
Confidence	Usually not core interface	First-class decision signal
Calibration	Task-dependent	Core design goal
Routing	Possible	Natural use case
Guardrails	Possible	Natural use case
Writing	Excellent use case	Not primary purpose
Deep reasoning	Stronger fit	Not primary purpose
Hallucination	Can generate unsupported text	Type safety reduces invalid output space
Semantic mistakes	Yes	Still possible

---

41. The biggest paradigm shift

Traditional AI architecture:

```
            AI
             ↓
      Generate Text
             ↓
    Parse the Text
             ↓
     Extract Decision
             ↓
       Execute Code
```

Decision-model architecture:

```
            AI
             ↓
     Typed Decision
             ↓
       Execute Code
```

[KEY] Remember this sentence

> *The model becomes a system primitive instead of only being a text-generation feature.*

---

42. AI as a system primitive

Traditional:

```
Application
    ↓
LLM API
    ↓
Text
    ↓
Application parses text
```

New conceptual approach:

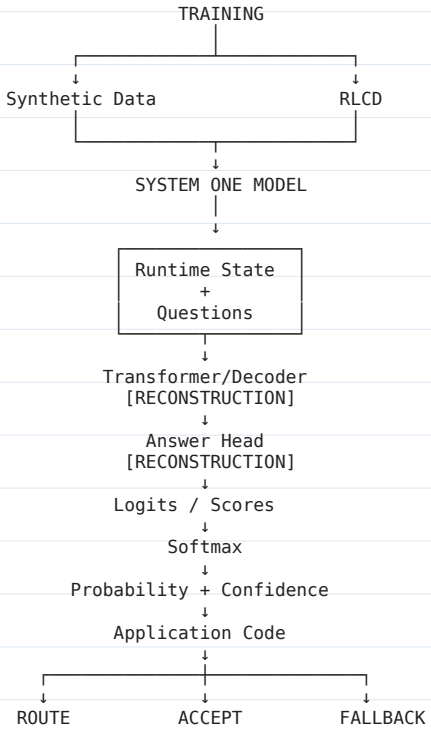
```
Application
    ↓
Decision Model
    ↓
Typed signal
    ↓
Application logic
```

This is particularly useful for:

```
routing
classification
verification
guardrails
scoring
moderation
real-time decisions
```

---

43. Full architecture — memorize this



---

44. One very important distinction

Model prediction

```
"I think A."
```

Calibrated decision system

```
"A"

Probability:
A = 0.82
```

```
B = 0.13
C = 0.05

Confidence:
0.82
```

Now software can establish:

```
if confidence > threshold:
      automate
else:
      fallback
```

That's why confidence is so important.

---

45. Limitations [KEY]

Don't blindly believe “breakthrough”.

Study these limitations:

1. Not a universal LLM replacement

It isn't intended to replace:

```
creative writing
long explanations
open-ended generation
general coding
complex reasoning
```

2. Can still be wrong

```
Correct schema
+
Wrong decision
=
still wrong
```

3. Calibration is statistical

```
80% confidence
≠
guaranteed 80% correctness for this one case
```

4. Internal architecture is not completely public

Therefore:

```
official facts
      ≠
reverse engineering
```

5. Benchmarks are workload-dependent

Always test:

```
Latency
Accuracy
Calibration
Cost
Throughput
Fallback rate
```

on your own data.

---

46. [IMPORTANT] Interview Questions

Q1. What is Jev?

Answer:

> *Jev is TypeSafe AI's System One decision model designed for fast structured decisions. Instead of generating text token by token like a traditional autoregressive LLM, it evaluates a state against typed questions and returns structured decisions with probabilities and confidence.*

Q2. Is Jev an LLM?

Say carefully:

> *It is a decision model rather than a conventional autoregressive language-generation model. TypeSafe positions it as complementary to generative LLMs.*

Q3. What is System One?

> *System One is the fast decision layer. It can handle routing, classification, scoring, verification and guardrails, while a generative model can handle deeper reasoning or text generation.*

Q4. What is parallel sampling?

> *Instead of generating one output token after another, the decision model can evaluate multiple requested decisions from shared state in parallel.*

Q5. What is RLCD?

> *RLCD stands for Reinforcement Learning for Calibrated Decisions. It is TypeSafe's term for training aimed at producing well-calibrated decision probabilities.*

Q6. Can Jev hallucinate?

Best answer:

> *Its schema-constrained interface can prevent invalid output types, but that doesn't mean it can never make a semantic error. It can still choose the wrong allowed option.*

Q7. What is calibration?

> *Calibration means predicted confidence should correspond to actual observed correctness over a population. For example, among predictions assigned 80% confidence, a calibrated model should be correct approximately 80% of the time.*

Q8. Jev vs ChatGPT?

Don't answer:

X "Jev is better."

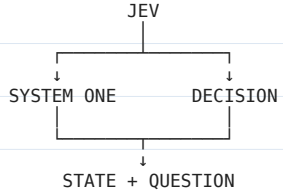
Answer:

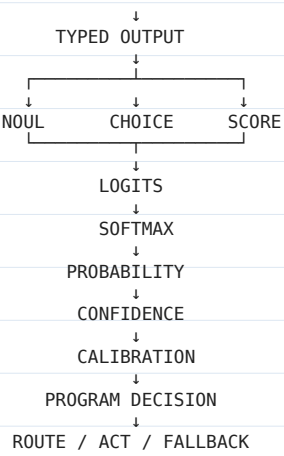
> *They target different workloads. A ChatGPT-style LLM is designed for language generation and reasoning, while Jev is designed around structured decisions and fast programmatic control.*

---

47. Final memory map

Write this at the last page of your notebook:





[KEY] 10 things to memorize first

- *Jev = System One decision model*
- *LLM = primarily text generation*
- *Jev = structured decision*
- *State = information being evaluated*
- *Question = decision the application asks*
- *Noul = yes/no*
- *Choice = select an option*
- *Score = ordered level/value*
- *Parallel sampling = multiple decisions from shared state*
- *Type safety ≠ semantic correctness*

And then:

```
Synthetic Data
      ↓
     RLCD
      ↓
System One Model
      ↓
State + Questions
      ↓
Answer Head*
      ↓
Softmax
      ↓
Probability + Confidence
      ↓
Code / Action

* lecture/reconstruction terminology
```

This is the core notebook version I'd recommend you learn before going into the deeper mathematics, transformer internals, KV-cache implications, decoder architecture, RLCD mechanics, and implementation.